

Desmos Custom Function Library User Manual

Desmos Custom Function Library User Manual (rd-261001)

Link: <https://www.desmos.com/geometry/hsi2zipg3w>

Table of Contents

1. Introduction and Conventions
 - 1.1 Background and Scope of Application
 - 1.2 Type-Notation Conventions
 - 1.3 Desmos Native Syntax Reference
 - 1.4 General Notes
2. Function Reference
 - 2.1 Geometry Drawing Functions
 - 2.2 Particle-System Functions
 - 2.3 Numeric Utility Functions
 - 2.4 Random-Number Utility Functions
 - 2.5 List Utility Functions
 - 2.6 Easing and Animation Functions
3. Advanced Function Reference
 - 3.1 Multi-Segment Sequenced Animation m_2
 - 3.2 Matrix Reshaping z_h
 - 3.3 List Mapping m_s
4. General Usage Precautions
5. Appendix
 - 5.1 f_l Easing-Index Look-Up Table

1 Introduction and Conventions

1.1 Background and Scope of Application

This document serves as the user manual for the custom-function library inside a Desmos notebook. This library encapsulates: arbitrary-order Bézier curves, oriented polar-coordinate transformations, circle-parameter utilities, a particle system, list CRUD utilities, random-value

generation, easing interpolation, multi-segment sequenced animation, matrix reshaping, and list-template mapping.

Document boundary: Explains purpose, parameter semantics, calling patterns, and input constraints; mathematical derivations of internal implementations are not covered.

Important Distinction:

- Desmos native functions: Built-in to the Desmos platform and available in any notebook, e.g. `sort()`, `random(n,seed)`, `nCr()`, `mod()`;
- Library custom functions (short subscripted names such as c_u, m_o): Pre-defined only in the notebook linked above, or usable only after copying appendix expressions into your own notebook.

1.2 Type-Notation Conventions

Notation	Meaning
Number	Single real number
Point	Plane coordinate point (x,y)
List[Number]	List composed of real numbers
List[Point]	List composed of plane points

Additional semantic conventions:

1. Time and angle parameters are numeric values;
2. Random seed is numeric: identical seed inputs produce fully reproducible random sequences;
3. List indices and count parameters are numeric; Desmos list indices start at 1; index 0 does not exist.

1.3 Desmos Native Syntax Reference

This library heavily relies on the following platform-native capabilities — these are not library-defined functions.

1. Two-argument sort $sort(a,b)$ **【Native】**
Re-orders list a synchronously according to ascending order of list b , and returns the re-ordered a .

Hard constraint: a, b must have exactly equal lengths; otherwise an error or empty result occurs.
Example: $sort([1, 2, 3, 4, 5, 6], [1, 1, 4, 5, 1, 4]) = [1, 2, 5, 3, 6, 4]$

2. Seeded random list $random(a,b)$ 【Native】

Returns a list of length a ; each element lies in $[0, 1)$; the second parameter b is the random seed.

Zero-argument `random()` is seed-uncontrolled and yields new random values on every invocation.

3. Rounding to decimal places $round(a,b)$ 【Native】

Returns a rounded to b decimal places.

4. Piece-wise expression {condition:value₁,defaultValue} 【Native】

Syntax: {condition1:valueA,condition2:valueB,fallbackValue}; matches top-to-bottom; returns the value for the first satisfied condition; returns the final fallback value if no condition holds.

5. Boolean-list index filtering $A[B=1]$ 【Native】

Constructs a new list containing elements from A at positions where $B = 1$; positions where B equals 0, 2, or any other number are filtered out.

Example: $A = [1, 2, 3, 4, 5, 6]$, $B = [1, 1, 4, 5, 1, 4]$, then $A[B = 1] = [1, 2, 5]$ 。

6. List slicing $A[a\cdots b]$ 【Native】

Extracts the sub-list from index a through b (both endpoints inclusive).

7. Sequence concatenation $join()$ 【Native】

$join(L1,L2,L3\cdots)$ concatenates multiple lists or for-iterated sequences. $c0.join(...)$ appearing in library code is Desmos dot-call syntactic sugar equivalent to $join(c0, ...)$.

8. Point member access $.x / .y$ 【Native】

Calling $.x / .y$ on a point or point-list extracts horizontal / vertical coordinates; for a point-list it returns a numeric-value list.

Pattern to reconstruct a point-list after processing numeric-value lists: $(X[i], Y[i])$ for $i=[1\cdots X.count()]$

9. Modulo operation $mod(m,n)$ 【Native】

Computes the remainder of m divided by n ; supports decimals and negative numbers.

Example: $mod(5, 2) = 1, mod(5.1, 2) = 1.1$ 。

10. Binomial coefficient $nCr(n,k)$ 【Native】 , used internally for Bézier evaluation.

1.4 General Notes

1. All list indices start at 1; index 0 must never be used as a list index.

2. This library contains no built-in fault tolerance: out-of-bounds slicing, illegal indices, or wrong parameter types directly yield empty output or errors; callers must guarantee valid inputs.

3. Point-lists cannot be passed directly into functions expecting numeric-value-lists; always split using .x, .y into separate numeric-value-lists, process them, then reassemble into a point-list.
4. There is no f_0 function in the original manuscript; old Word-document formula entries are transcription errors and everywhere should read f_l .

2 Function Reference

Mark **【Library-Defined】** indicates functions available only after loading library expressions; return type is noted for each entry.

2.1 Geometry Drawing Functions

2.1.1 General-order Bézier Curve $c_u(p_{ce}, t_{cu})$ **【Library-Defined】**

Purpose: Evaluate an arbitrary-order Bézier curve; the number of control points automatically determines curve order.

$$c_u(p_{ce}, t_{cu}) = \sum_{n=0}^{p_{ce}.count()-1} nCr(p_{ce}.count()-1, n) (1-t_{cu})^{(p_{ce}.count()-n-1)} t_{cu}^n (p_{ce}[n+1])$$

Parameter	Type	Meaning
p_{ce}	List[Point]	All Bézier control points, ordered along drawing direction
t_{cu}	Number	Curve parameter, normally $t_{cu} \in [0, 1]$; 0 selects start-point, 1 selects end-point

Return value: Point; coordinates on the curve for parameter t_{cu} .

Minimal example: second-order Bézier with control-points $p_{ce} = [(0, 0), (2, 3), (5, 1)]$; call $c_u(p_{ce}, 0.5)$ to obtain the mid-point on the curve.

2.1.2 Piecewise Cubic Bézier Curve $c_{u2}(l_0, t_0)$ **【Library-Defined】**

Purpose: Emulate pen-tool multi-segment cubic Bézier; every consecutive four points constitute one cubic-Bézier segment; maps global time to each individual segment for evaluation.

$$c_{u2}(l_0, t_0) = c_u(l_0[3i-2 \dots 3i+1], c_{ut}(0, t_0-i+1, 1)) \text{ for } i = \left\lceil 1 \dots \text{floor}\left(\frac{l_0.count()}{3}\right) \right\rceil$$

--	--	--

Parameter	Type	Meaning
l_0	List[Point]	Full list of control-points; every four consecutive points define one cubic-Bézier segment; user must guarantee list-length is a multiple of 4
t_0	Number	Global time parameter driving sampling-evaluation for the full curve

Return value: List[Point]; sequence of sample-points for the complete piecewise Bézier curve.

Pitfall: Internally calls clamping function c_{ut} ; loop upper bound uses $\text{floor}(l_0.\text{count}()/3)$, this is an intrinsic library limitation; slicing out-of-bounds occurs if input length is not a multiple of 4.

2.1.3 Polar-Coordinate Transformation $j_z(t_{jz}, f_{jz}, x_{jz}, y_{jz})$ 【Library-Defined】

Purpose: Convert polar-coordinates to Cartesian-coordinates with phase offset: angle0 points straight upward; increasing angle rotates clockwise; supports global coordinate offset.

$$j_z(t_{jz}, f_{jz}, x_{jz}, y_{jz}) = \left(f_{jz} \sin\left(-t_{jz} + \frac{\pi}{2}\right) + x_{jz}, f_{jz} \cos\left(-t_{jz} + \frac{\pi}{2}\right) + y_{jz} \right)$$

Parameter	Type	Meaning
t_{jz}	Number	Polar angle (radians)
f_{jz}	Number	Polar radius
x_{jz}	Number	Horizontal offset for output point
y_{jz}	Number	Vertical offset for output point

Return value: Point.

2.1.4 Circle Parametric Equation $r_o(x_0, y_0, r_0, t_0)$ 【Library-Defined】

Purpose: Retrieve point on circumference corresponding to given angle; angle0 points straight upward; increasing t_0 rotates clockwise.

$$r_o(x_0, y_0, r_0, t_0) = (r_0 \sin t_0 + x_0, r_0 \cos t_0 + y_0)$$

Parameter	Type	Meaning

x_0	Number	Circle-center horizontal coordinate
y_0	Number	Circle-center vertical coordinate
r_0	Number	Circle radius
t_0	Number	Angle parameter (radians)

Return value: Point.

2.2 Particle-System Functions

2.2.1 Region Particle Spawning $l_z(a_0,b_0,n_0,t_0)$ 【Library-Defined】

Purpose: Generate n_0 randomly-positioned particles within a rectangle defined by two diagonal corner-points.

$$l_z(a_0,b_0,n_0,t_0)=(r_a(a_0.x,b_0.x,n_0,t_0,3),r_a(a_0.y,b_0.y,n_0,t_0+1,3))$$

Parameter	Type	Meaning
a_0	Point	One diagonal corner-point of rectangle
b_0	Point	Opposite diagonal corner-point of rectangle
n_0	Number	Total particle count
t_0	Number	Random seed

Return value: List[Point]; n_0 random points inside the rectangle.

⚠Pitfall: Does **not** automatically perform min/max correction; you must guarantee $a_0.x < b_0.x, a_0.y < b_0.y$, otherwise inverted / invalid intervals are produced.

2.2.2 Region Particle Motion $l_{zm}(a_0,b_0,c_0,d_0,n_0,t_0,s_0)$ 【Library-Defined】

Purpose: Particles move over time; crossing rectangle boundaries wraps coordinates to the opposite side (toroidal / cyclic boundary behaviour).

$$l_{zm}(a_0,b_0,c_0,d_0,n_0,t_0,s_0)=(\begin{matrix} m_o(a_0.x,(p_0+t_0c_0z_0).x,b_0.x), \\ m_o(a_0.y,(p_0+t_0c_0z_0).y,b_0.y) \end{matrix})$$

with

$$p_0=l_z(a_0,b_0,n_0,s_0),$$

$$z_0=sort(r_a(1,d_0+1,n_0,s_0+2,2))$$

Parameter	Type	Meaning
a_0	Point	Lower-bound corner-point of simulation region
b_0	Point	Upper-bound corner-point of simulation region
c_0	List[Number]	Base velocity coefficient for each particle; ideally length equals particle count n_0
d_0	Number	Upper bound for velocity randomization
n_0	Number	Particle count
t_0	Number	Time parameter
s_0	Number	Random seed for initial-position generation

Return value: List[Point]; positions of all particles for current time-step.

Note: z_0 is internally-generated sorted random velocity-multiplier; users do not need to manipulate it. If c_0 length differs from n_0 , Desmos broadcasts list elements, producing anomalous particle velocities.

2.3 Numeric Utility Functions

2.3.1 Clamp $c_{ut}(a_{cut}, b_{cut}, c_{cut})$ 【Library-Defined】

Purpose: Restrict numeric value to interval $[a_{cut}, b_{cut}]$.

$$c_{ut}(a_{cut}, b_{cut}, c_{cut}) = \max(a_{cut}, \min(b_{cut}, c_{cut}))$$

Parameter	Type	Meaning
a_{cut}	Number	Interval lower bound
b_{cut}	Number	Interval upper bound
c_{cut}	Number	Input numeric value to be clamped

Return value: Number; result lies within $[a_{cut}, b_{cut}]$.

Example: $c_{ut}(0, 1, 1.5) = 1$; $c_{ut}(0, 1, -0.2) = 0$.

2.3.2 Interval Normalization $t_c(t_0, t_{end}, t_{now})$ 【Library-Defined】

Purpose: Compute relative fraction of current value inside given interval; output is clamped to $[0, 1]$; commonly used to convert time into interpolation coefficient.

$$t_c(t_0, t_{end}, t_{now}) = c_{ut}\left(0, \frac{t_{now} - t_0}{t_{end} - t_0}, 1\right)$$

Parameter	Type	Meaning
t_0	Number	Interval start point
t_{end}	Number	Interval end point
t_{now}	Number	Current timestamp / input value

Return value: Number; range $[0, 1]$.

2.3.3 Interval Wraparound $m_o(a_0, b_0, c_0)$ 【Library-Defined】

Purpose: Use modulo arithmetic to wrap arbitrary real numbers back into interval $[a_0, c_0]$ for cyclic-boundary behaviour.

$$m_o(a_0, b_0, c_0) = mod(b_0 - a_0, c_0 - a_0) + a_0$$

Parameter	Type	Meaning
a_0	Number	Interval lower bound
b_0	Number	Numeric value to wrap
c_0	Number	Interval upper bound

Return value: Number; lies inside $[a_0, c_0]$.

Example: $m_o(0, 5, 2) = 1$, wraps 5 into interval $[0, 2]$ yielding 1.

2.3.4 Linear Interpolation $l(s_{start}, e_{end}, t_0)$ 【Library-Defined】

Purpose: Perform linear interpolation between start-value and end-value; $t_0 = 0$ returns start-value, $t_0 = 1$ returns end-value.

$$l(s_{start}, e_{end}, t_0) = t_0(e_{end} - s_{start}) + s_{start}$$

--	--	--

Parameter	Type	Meaning
s_{start}	Number	Interpolation starting value
e_{end}	Number	Interpolation target ending value
t_0	Number	Interpolation coefficient, normally $t_0 \in [0, 1]$

Return value: Number.

2.4 Random-Number Utility Functions

2.4.1 Single Random Number $f_r(x)$ 【Library-Defined】

Purpose: Return random value in $[0, 1)$; parameter x is a placeholder and participates in no computation.

$$f_r(x) = random()$$

Parameter	Type	Meaning
x	Number	Formal placeholder argument, has no functional effect

Return value: Number; range $[0, 1)$.

⚠Important: Internally calls zero-argument random(), which is seed-uncontrolled; use r_a when reproducible random sequences are required.

2.4.2 Random-Number List $r_a(a_0, b_0, n_0, t_0, r_0)$ 【Library-Defined】

Purpose: Generate list of specified length with values inside given numeric interval, rounded to specified decimal-place count.

$$r_a(a_0, b_0, n_0, t_0, r_0) = round(l(a_0, b_0, random(n_0, t_0)), r_0)$$

Parameter	Type	Meaning
a_0	Number	Lower bound for random-value interval
b_0	Number	Upper bound for random-value interval

n_0	Number	Output list length
t_0	Number	Random seed
r_0	Number	Decimal-place count for rounding

Return value: List[Number]; length n_0 , elements lie within $[a_0,b_0]$.

2.4.3 Diverging Sequence $t_e(a_0,b_0,n_0,c_0)$ 【Library-Defined】

Purpose: Generate diverging numeric sequence incorporating linear-step plus quadratic-acceleration term.

$$t_e(a_0,b_0,n_0,c_0) = a_0 + \left(b_0i + \frac{1}{2}c_0b_0i^2\right) \text{ for } i = [0...n_0]$$

Parameter	Type	Meaning
a_0	Number	Sequence starting value
b_0	Number	Base step size
n_0	Number	Iteration count
c_0	Number	Quadratic-term coefficient controlling divergence acceleration; $c_0 > 0$ makes divergence speed increase over sequence

Return value: List[Number]; total length n_0+1 .

Note: i is iterator variable, not a list index.

2.5 List Utility Functions

⚠This entire group accepts **only numeric-value-lists**. For point-lists: split via .x, .y, perform computations separately, then reassemble point-list.

2.5.1 List Insert $i_c(A_0,a_0,B_0)$ 【Library-Defined】

Purpose: Fully insert sub-list B_0 at index a_0 inside source-list A_0 .

$$i_c(A_0,a_0,B_0) = \{i < a_0 : A_0[i], \{a_0 \leq i < a_0 + B_0.count() : B_0[i-a_0+1], A_0[i-B_0.count()]\} \} \text{ for } i = [1...A_0.count() + B_0.count($$

Parameter	Type	Meaning
A_0	List[Number]	Original source list
a_0	Number	Insert-position index; $a_0 = 1$ inserts at very front; $a_0 = A_0.count() + 1$ appends to end
B_0	List[Number]	Sub-list to be inserted

Return value: List[Number].

Pitfall: Out-of-bounds a_0 causes errors; no fault-tolerance.

2.5.2 List Delete $d_{el}(A_0, a_0, b_0)$ 【Library-Defined】

Purpose: Delete b_0 consecutive elements starting at index a_0 .

$$d_{el}(A_0, a_0, b_0) = \{i < a_0 : A_0[i], A_0[i + b_0]\} \text{ for } i = [1 \dots A_0.count() - b_0]$$

Parameter	Type	Meaning
A_0	List[Number]	Original source list
a_0	Number	Starting index for deletion
b_0	Number	Number of elements to delete

Return value: List[Number].

Behaviour: If requested deletion range exceeds list tail, deletion stops at end-of-list.

2.5.3 List Replace $r_p(A_0, a_0, B_0)$ 【Library-Defined】

Purpose: Overwrite source-list starting from index a_0 using list B_0 ; if B_0 extends past original-list length, append new elements expanding output-list.

$$r_p(A_0, a_0, B_0) = \{a_0 \leq i \leq a_0 + B_0.count() - 1 : B_0[i - a_0 + 1], A_0[i]\} \text{ for } i = [1 \dots \max(A_0.count(), B_0.count() + a_0 - 1)]$$

Parameter	Type	Meaning
A_0	List[Number]	Original source list
a_0	Number	Starting index for replacement

B_0	List[Number]	New-data list used for overwriting
-------	--------------	------------------------------------

Return value: List[Number].

2.5.4 Find Index of Nearest Value $s_e(l_0, a_0)$ 【Library-Defined】

Purpose: Return index of element inside list l_0 whose numeric value is closest to target a_0 .

$$s_e(l_0, a_0) = \text{sort}([1 \dots l_0.\text{count}()], |l_0 - a_0|)[1]$$

Parameter	Type	Meaning
l_0	List[Number]	List to search within
a_0	Number	Reference target value

Return value: Number.

Behaviour: When multiple elements have equal minimal distance, returns smallest-index match.

2.5.5 Find All Matching Indices $f_d(l_0, a_0)$ 【Library-Defined】

Purpose: Return list containing every index position where list-element exactly equals target a_0 .

$$f_d(l_0, a_0) = b_0[l_0 = a_0] \text{ with } b_0 = [1 \dots l_0.\text{count}()]$$

Parameter	Type	Meaning
l_0	List[Number]	List to search within
a_0	Number	Target value for exact-equality match

Return value: List[Number]; collection of all indices satisfying equality condition.

⚠Pitfall: Performs strict floating-point exact-equality test; floating-point rounding error may cause matches to fail.

2.6 Easing and Animation Functions

2.6.1 Easing-Function Collection $f_i(x)$ 【Library-Defined】

Purpose: Input $x \in [0, 1]$, compute multiple easing-curve outputs simultaneously and return them as a single list; select individual easing-result via list indexing.

$$f_l(x) = \left\{ x \leq 0 : 0, x \geq 1 : 1, \left[x, \text{floor}(x), 1 - \cos\left(\frac{x\pi}{2}\right), \sin\left(\frac{x\pi}{2}\right), \dots \right] \right\}$$

Parameter	Type	Meaning
x	Number	Easing input; recommended domain $[0, 1]$

Return value: List[Number].

Calling example: $f_l(x)[w]$, where w is easing-index; retrieves output of w -th easing curve. Complete index table in appendix 5.1.

2.6.2 Bounce Easing Sub-Function $f_{l1}(x)$ 【Library-Defined】

Purpose: Piecewise implementation of bounce easing; already included as one entry in output-list of f_l ; may also be invoked standalone.

$$f_{l1}(x) = \begin{cases} x < \frac{1}{2.75}: & 7.5625x^2, \\ x < \frac{2}{2.75}: & 7.5625\left(x - \frac{1.5}{2.75}\right)^2 + 0.75, \\ x < \frac{2.5}{2.75}: & 7.5625\left(x - \frac{2.25}{2.75}\right)^2 + 0.9375, \\ & 7.5625\left(x - \frac{2.625}{2.75}\right)^2 + 0.984375 \end{cases}$$

Parameter	Type	Meaning
x	Number	Easing input; recommended domain $x \in [0, 1]$

Return value: Number.

2.6.3 Single-Segment Eased Animation $m(t_{m0}, v_{m0}, t_{m1}, v_{m1}, t_{now}, w_{ay})$ 【Library-Defined】

Purpose: Perform one-dimensional interpolated animation with specified easing-index over one time interval.

$$m(t_{m0}, v_{m0}, t_{m1}, v_{m1}, t_{now}, w_{ay}) = l(v_{m0}, v_{m1}, f_l(t_c(t_{m0}, t_{m1}, t_{now}))) [w_{ay}]$$

Parameter	Type	Meaning
t_{m0}	Number	Start-timestamp of this animation segment
v_{m0}	Number	Initial numeric value for this segment

t_{m1}	Number	End-timestamp of this animation segment
v_{m1}	Number	Target final numeric value for this segment
t_{now}	Number	Global current time
w_{ay}	Number	Easing index, corresponds to list subscript inside $f_l(\cdot)$

Return value: Number; animation output value for current timestamp.

⚠ Only implements one-dimensional numeric animation! To animate points: call m separately for x-coordinate and y-coordinate, then assemble as point(mx, my).

3 Advanced Function Reference

3.1 Multi-Segment Sequenced Animation $m_2(l_m, t_{now})$ 【Library-Defined】

Purpose: Sequence multiple animation segments chronologically; supply configuration-list and current-time, obtain one-dimensional animated output value for current moment.

$m_2(l_m, t_{now}) = \{s_m \leq 5 : m(l_m[1], l_m[2], l_m[3], l_m[4], t_{now}, l_m[5]), \{t_{now} < l_m[s_m - 4] : m_2(l_m[1 \dots s_m - 5], t_{now}), l(l_m[s_m - 3], l_m[s_m - 1], f_l(t_c(l_m[s_m - 4], l_m[s_m - 2], t_{now})))(l_m[s_m])\}\}$ with $s_m = l_m.count()$

Parameter	Type	Meaning
l_m	List[Number]	Animation-configuration list; every five numeric values describe one animation segment; total list-length must be multiple of 5
t_{now}	Number	Global current time

Return value: Number; animated output value for current timestamp.

3.1.1 l_m Configuration Format

Each segment occupies five consecutive numeric entries:

--	--

Position within-segment	Meaning
1st entry	Segment start-time t_{m0}
2nd entry	Segment initial-value v_{m0}
3rd entry	Segment end-time t_{m1}
4th entry	Segment target final-value v_{m1}
5th entry	Easing index w_{ay} (subscript for f_l)

3.1.2 Runtime Logic

Function performs time-check starting from last animation-segment in configuration-list:

1. Current time precedes segment start-time $\rightarrow$ recursively evaluate earlier preceding segments;
2. Current time falls within segment time-window $\rightarrow$ perform eased interpolation using parameters for this segment and return result;

When t_{now} exceeds end-timestamp of all segments: t_c clamps output to 1; output holds constant at final-value of last segment.

3.1.3 Examples

1) Single-segment animation: time 0-2, numeric value animates $0 \rightarrow 10$, easing-index 2

$$l_m = [0, 0, 2, 10, 2]$$

2) Two-segment seamless sequence: segment-1 $0 \rightarrow 2$ ($0 \rightarrow 10$, easing 2); segment-2 $2 \rightarrow 5$ ($10 \rightarrow 30$, easing 3)

$$l_m = [0, 0, 2, 10, 2, 2, 10, 5, 30, 3]$$

3.1.4 Hard Constraints

l_m must be purely numeric-value-list; points are forbidden; total length must be multiple of 5;

2. Timestamps must be monotonically non-decreasing; subsequent-segment start-time $\geq$ preceding-segment end-time; equality yields seamless join;
3. To avoid value-jumps: segment initial-value should equal final-value of prior-segment;
4. Easing-index must be valid subscript for f_l ;
5. Overlapping time-intervals raise no runtime error but produce corrupted animation output.

Point-animation handling: prepare separate l_m configurations for x-axis and y-axis; call m_2 twice independently, then combine into point.

3.2 Matrix Reshaping $z_h(l_0, a_0)$ 【Library-Defined】

Purpose: Pack one-dimensional list into matrix with specified row-count; transpose matrix, then flatten back to one-dimensional list; frequently used for grid-point coordinate re-ordering.

$$z_h(l_0, a_0) = \text{sort} \left(l_0, \left(j \text{ for } j = \left\lceil 1 \dots \frac{l_0.\text{count}()}{a_0} \right\rceil, i = [1 \dots a_0] \right) \right)$$

Parameter	Type	Meaning
l_0	List[Number]	Source one-dimensional data-list
a_0	Number	Target matrix row-count (not column-count)

Return value: List[Number].

Computation Rules

1. Fill matrix of a_0 rows row-wise using elements from l_0 ; column-count $= l_0.\text{count}() / a_0$;
2. Traverse matrix column-by-column and concatenate elements to produce output one-dimensional list.

⚠Hard constraint: Source-list length must be divisible by a_0 ; otherwise sort-key length mismatch yields invalid output.

Valid Examples

Example 1: $z_h([1, 2, 3, 4, 5, 6], 2)$

Row-wise fill into 2-row × 3-column matrix:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \text{ read column-wise output: } [1, 4, 2, 5, 3, 6]$$

Example 2: $z_h([1, 2, 3, 4, 5, 6], 3)$

Row-wise fill into 3-row × 2-column matrix:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}, \text{ read column-wise output: } [1, 3, 5, 2, 4, 6]$$

Example 3: $z_h([1, 2, 3, 4, 5, 6, 7, 8, 9], 3)$

Output: $[1, 4, 7, 2, 5, 8, 3, 6, 9]$

3.3 List Mapping $m_s(l_0, a_0, b_0, c_0)$ 【Library-Defined】

Purpose: Partition source-list into fixed-size groups; apply identical mapping-template to every group to generate output fragments; concatenate fragments for final-output-list; template marker0 reads from fallback-pool instead of source-list.

$$m_s(l_0, a_0, b_0, c_0) = \left(\{b_0[j] = 0 : d_0[(i-1)p_0 + j], l_0[(i-1)a_0 + b_0[j]]\} \text{ for } j = [1 \dots p_0], i = \left\lceil \frac{l_0.\text{count}()}{a_0} \right\rceil \right) \text{ with } d_0 = c_0.\text{join}\left(0 \text{ for } i = \left\lceil \frac{l_0.\text{count}()}{a_0} \right\rceil b_0.\text{count}() - c_0.\text{count}()\right), p_0 = b_0.\text{count}()$$

Parameter	Type	Meaning
l_0	List[Number]	Source-data list
a_0	Number	Group-size: number-of-input-elements per group
b_0	List[Number]	Mapping-template; template-length equals output-element-count per group
c_0	List[Number]	Fallback source-data list for template slots marked 0

Return value: List[Number].

Template Semantics

Non-zero numeric entries inside b_0 : retrieve element at corresponding local-group index (local index starts 1);

When b_0 entry=0: do **not** read from source-list; read from fallback-pool d_0 ;

- Fallback-pool d_0 : pad c_0 with trailing zeros until total-output-length is satisfied;
- Output-slots marked 0 index into d_0 using their absolute global index inside final-output-list;
- When global-index exceeds original c_0 length, padded-zero value is retrieved;
- Total output-length=group-count $\times$ template-length; group-count $=\text{ceil}(l_0.\text{count}()/a_0)$.

⚠Pitfall: If source-list length is **not divisible by group-size** a_0 , final-group indexing goes out-of-bounds producing undefined values; caller must guarantee source-list length is multiple of a_0 .

Full Working Example

Call: $m_s([1 \dots 10], 2, [2, 0, 1], [-1 \dots -15])$
 $l_0 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$; group-size $a_0 = 2$; 5 total groups; template $b_0 = [2, 0, 1]$, every group outputs 3 entries; $c_0 = [-1, -2, \dots, -15]$.

Group-ID	Elements in-group	Template slot1 (take group-local#2)	Template slot2 (read fallback-pool)	Template slot3 (take group-local#1)	Output for-group
1	[1, 2]	2	−2	1	[2, −2, 1]
2	[3, 4]	4	−5	3	[4, −5, 3]
3	[5, 6]	6	−8	5	[6, −8, 5]
4	[7, 8]	8	−11	7	[8, −11, 7]
5	[9, 10]	10	−14	9	[10, −14, 9]

Concatenated final output: [2, −2, 1, 4, −5, 3, 6, −8, 5, 8, −11, 7, 10, −14, 9]

4 General Usage Precautions

1. Type-matching: numeric-functions accept numeric inputs; list-functions accept numeric-value-lists; point-functions accept points / point-lists; passing wrong-type inputs causes errors.
2. All list indices start at 1; never use 0 as list index.
3. No built-in fault-tolerance: out-of-bounds slicing, illegal indices, non-divisible lengths are **not auto-repaired**; caller validates inputs beforehand.
4. Point-list processing workflow: Point-list → split-to-two numeric-value-lists via .x / .y → invoke list-utility-functions separately → reassemble point-list with (X[i],Y[i]) for i=[1⋯ X.count()].
5. Random seeds: r_a supports seeds for reproducible random-sequences; f_r is seed-uncontrolled.
6. Easing-index constraints: $f_l(x)[w]$ index w cannot exceed total element-count returned by f_l ; refer to appendix index-table.
7. Most library-functions are pure-functions: identical inputs always yield identical outputs; only l_z , l_{zm} , r_a , f_r incorporate random-behaviour.

5 Appendix

5.1 f_l Easing-Index Look-Up Table

Order matches original manuscript txt-source expression for f_l (indices start 1).

Easing references source: <https://easings.net> ; if discrepancies exist in this document, defer to the external webpage.

Indexw	Easing-effect description	Indexw	Easing-effect
1	Linear x	16	ease-out quint $1 - (1 - x)^5$
2	Constant	17	ease-in-out quint
3	ease-out sine $1 - \cos(\pi x/2)$	18	ease-in exponential 2^{10x-10}
4	ease-in sine $\sin(\pi x/2)$	19	ease-out exponential $1 - 2^{-10x}$
5	ease-in-out sine	20	ease-in-out exponential
6	ease-in quad x^2	21	ease-in-out circular in
7	ease-out quad $1 - (1 - x)^2$	22	ease-out circular out
8	ease-in-out quad	23	ease-in-out circular
9	ease-in cubic x^3	24	ease-in back
10	ease-out cubic $1 - (1 - x)^3$	25	ease-out back
11	ease-in-out cubic	26	ease-in-out back
12	ease-in quart x^4	27	ease-in elastic
13	ease-out quart $1 - (1 - x)^4$	28	ease-out elastic
14	ease-in-out quart	29	ease-in-out elastic
15	ease-in quint x^5	30	ease-out bounce ($1 - f_l(1 - x)$)
		31	ease-in bounce ($f_l(x)$)
		32	ease-in-out bounce

Hint: Directly evaluate `f_l(x).count()` within Desmos to inspect actual total list-length.